

CO4 - AT1

COMPARATIVE ANALYSIS

P. Deepika
192525026

Q1. A collision occurs in hashing when two or more keys produce the same hash-table index.

a) Clustering Effects.

Linear Probing suffers from Primary clustering.

For example: if Position 5, 6, 7 and 8 become occupied, a new element hashing to any of these location may have to search through this entire continuous block.

This creates large cluster, causing future searches and insertion to become slower.

b) Memory usage.

Linear probing stores all elements directly inside the hash table.

Therefore, it requires relatively little additional memory.

Separate chaining requires additional nodes and usually pointers/reference for the linked lists.

c) Performance under high load factor.

As a load factor increases, linear probing becomes inefficient because there are fewer empty positions and clusters become large.

for example -

$\alpha \leq 0.3$	Good performance
$\alpha \leq 0.5$	generally acceptable
$\alpha \approx 1$	Performance becomes poor.

d) Suitability for large-scale applications.

For large-scale application with frequent insertion and deletion and potentially unpredictable data size, separate chaining is generally more suitable.

Example.

- database indexing
- Symbol tables
- Caches
- Large dictionaries

linear probing can be faster in practice when:

- load factor is controlled.
- Table fits well in cache
- Memory efficiency is important

Q2. Merge sort vs Quick sort for linked list & CO

For a linked list containing 10 million nodes the choice of sorting algorithm is important because linked lists have different characteristics from arrays.

Factor	Merge sort	Quick sort
Average time	$O(n \log n)$	$O(n \log n)$
Worst-case time	$O(n \log n)$	$O(n^2)$
Best-case time	$O(n \log n)$	$O(n \log n)$
extra space	$O(\log n)$ recursive	$O(\log n)$ average
Linked-list suitability	excellent	Moderate.
Sequential access	very suitable	Less suitable.
Splitting	easy using slow	Partitioning is not comfortable.
Worst case guarantee	Yes	No, unless carefully implemented
Cache locality.	Good sequential access	Can be poorer on linked list
Stability	Stable	usually not stable

merge sort:

Merge Sort accesses nodes sequentially while merging, making it relatively suitable for linked lists.

however linked list nodes can be scattered in memory so, cache is still not as good as with arrays.

Quick sort:

→ selects a pivot and partitions element into smaller and larger groups.

For 10 million nodes, a badly balanced recursive Quick Sort could therefore create very large recursion depth.

Q2. Quadratic Probing vs Double hashing.

Both Quadratic Probing and double hashing are open-addressing techniques

When a collision occurs, they calculate another position instead of using a separate linked list.

Quadratic probing:

$$h(k), h(k) + 1^2, h(k) + 2^2, h(k) + 3^2, \dots$$

General formula:

$$h_i(k) = (h(k) + c_1 i + c_2 i^2) \bmod m$$

keys with same initial hash value follow the same probe sequence.

Double hashing.

$h_1(k)$ and $h_2(k)$

Probe sequence is

$$h_i(k) = [h_1(k) + i h_2(k)] \bmod m$$

Therefore, two keys that initially collide may follow different probe sequence if their second hash value differ

Factor.	Quadratic probing.	Double probing.
Primary clustering	Avoids	Avoids
Secondary clustering	Present	very low
Probe sequence	Quadratic	Based on second hash
Hash function	usually one	Two
collision resolution	good	very good
computation	simpler	Slightly more computation
Dense table	less suitable	more suitable
implementation	easier	more complex
Distribution.	Better than linear probing.	Usually best among open addressing schemes

24. Best case and worst case time complexity of sorting algorithm.

Algorithm	Best case	Average case
Bubble sort	$O(n)$	$O(n^2)$
Selection sort	$O(n^2)$	$O(n^2)$
Insertion sort	$O(n)$	$O(n^2)$
Merge sort	$O(n \log n)$	$O(n \log n)$
Quick sort	$O(n \log n)$	$O(n \log n)$
Heap sort	$O(n \log n)$	$O(n \log n)$

a) Practical signature.

Bubble-sort:

$O(n)$

When the array is already sorted and an optimized version detects no swaps.

Selection sort:

Selection sort always searches for the minimum element.

Therefore, even if the array is already sorted it still performs approximately $O(n^2)$.

b) impact on Algorithm selection.

The best-case complexity tells us how efficiently an algorithm can perform under favourable conditions.

The worst-case complexity tells us how badly it can perform under unfavourable conditions.

Q5. Recursive vs iterative quicksort.

Quicksort can be implemented in two major ways:

1. Recursive Quicksort
2. Iterative Quicksort.

Recursive Quicksort:

→ The recursive version calls itself after partitioning.

Quicksort(Array, low, high)

if $low < high$

Pivot = partition(Array)

Quicksort(Array, low, Pivot - 1)

Quicksort(Array, Pivot + 1, high)

For balanced partitions:

$O(\log n)$

Unbalanced partitions:

$O(n)$

Iterative quick sort

The iterative implementation eliminates function recursion by explicitly maintaining subarray ranges using a stack.

Push(low, high)

while stack is not empty:

Pop(low, high)

Partition()

Push(left subarray)

Push(right subarray)

For balanced partitions,

$O(\log n)$

Worst case can still reach

$O(n)$.

If both sides are handled poorly,